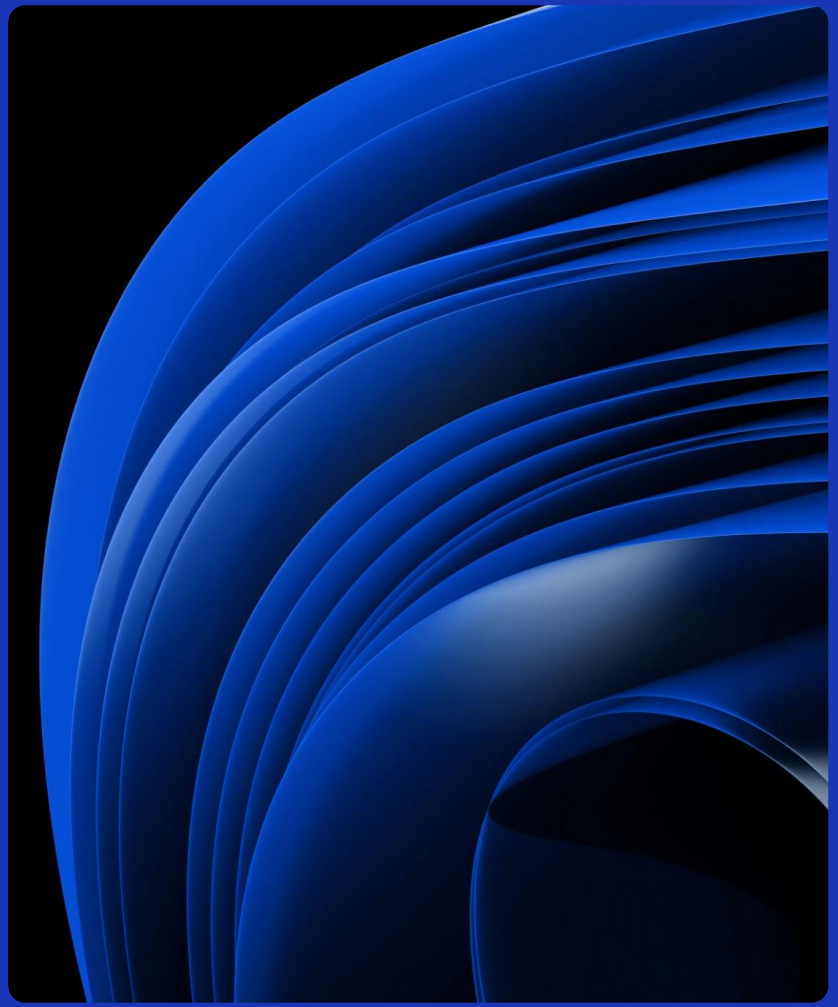
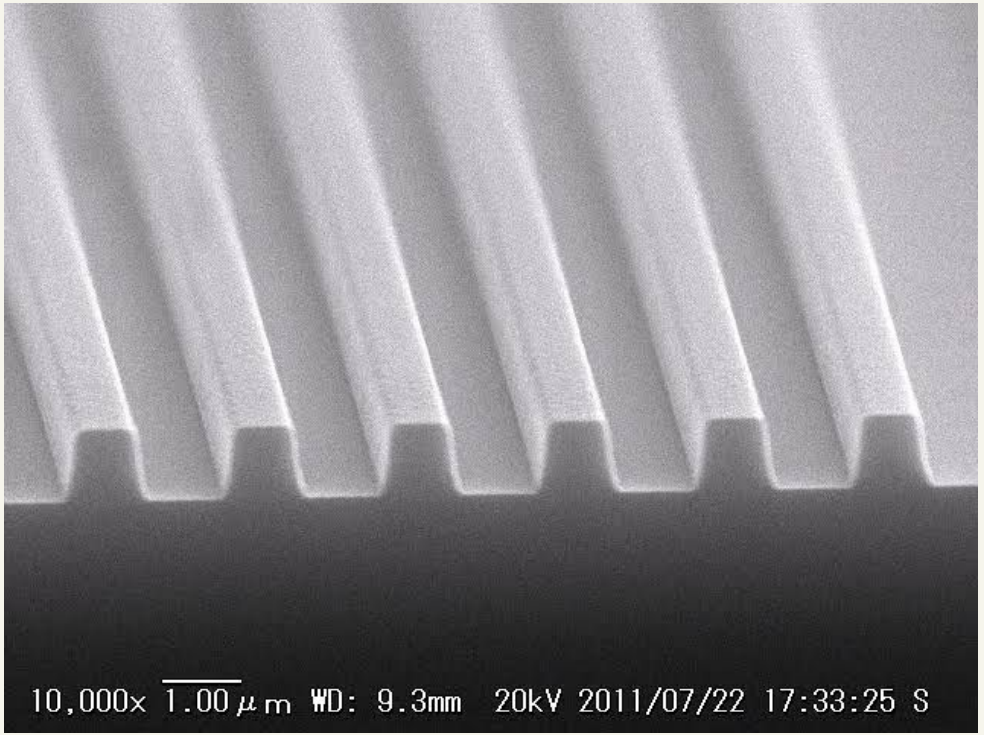
CUDA Floating Point Electron Beam Lithography Proximity Effect Correction



8.12.2026
George Trupiano, Andrew Radwan,
Ruben Parra Montenegro
Oakland University
ECE 5770

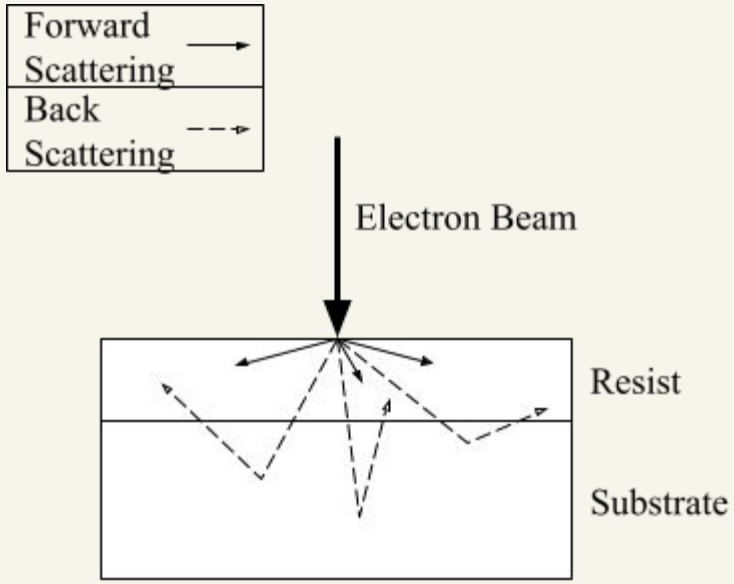
Electron Beam Lithography

- Process in which a focused beam of electrons is used to create nanoscale structures out of a resist (negative e-beam resist in our case)
- Achieves feature sizes optical lithography cannot reach (UV light)
- Used for photomask fabrication, prototyping, and cutting edge research



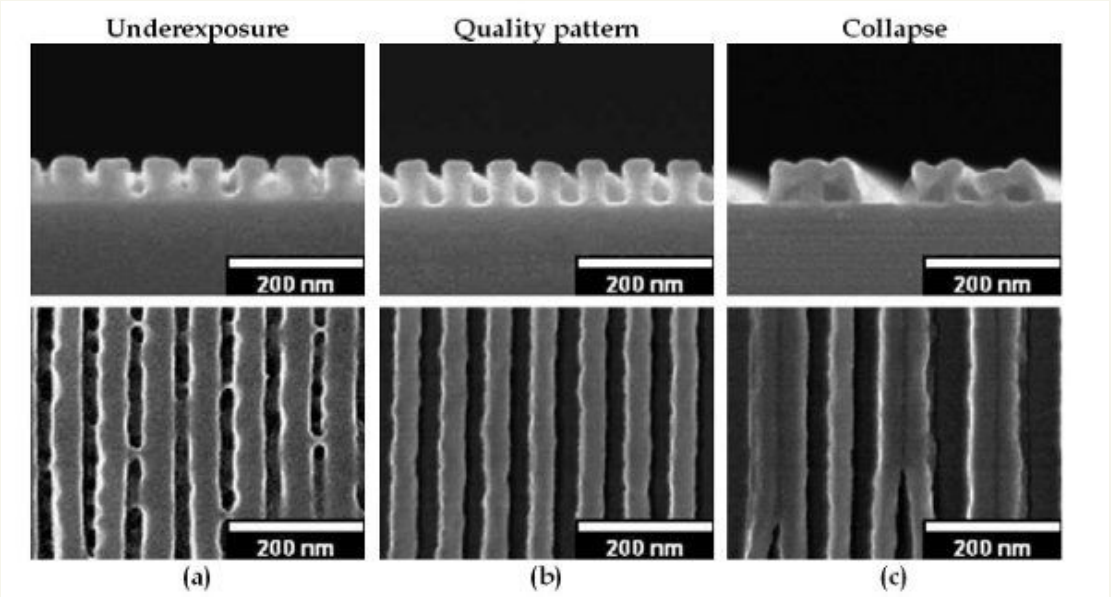
Problem: Electron Beam Scattering

- Upon contact with a resist, electrons scatter as they pass through exposing unwanted areas
- Forward Scattering: electron-electron interactions in resist scattering them
- Backward Scattering: electrons in substrate collide with atomic nuclei reflecting back up into resist



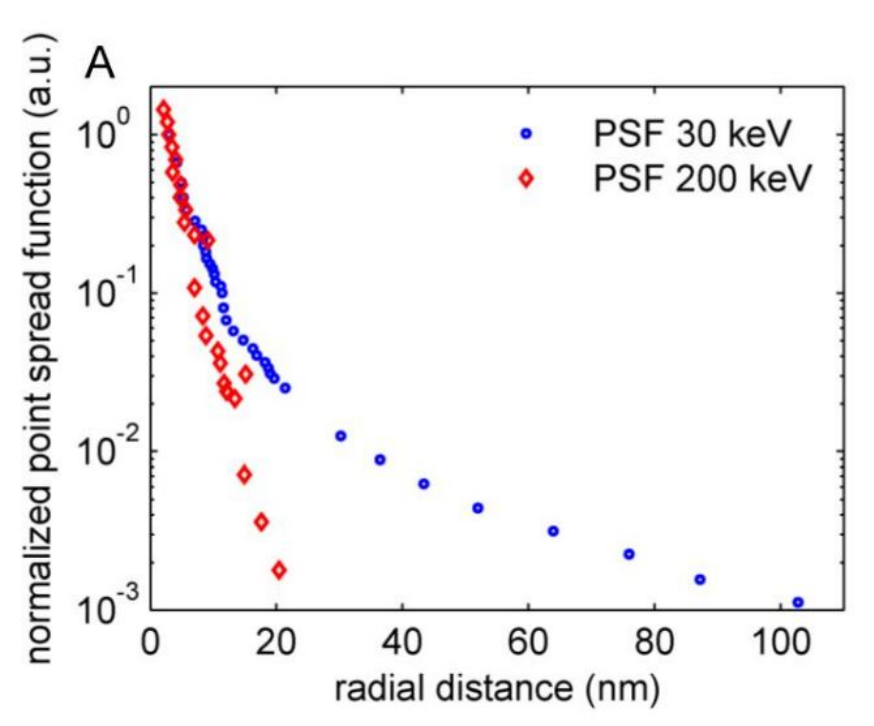
Problem: Electron Beam Scattering

- Quality results require the resist is exposed in the right locations for the right amount of time
- Dense regions of an IC become overexposed from forward and backward scattering
- While isolated regions turn out underexposed due to the same effect



Pushing the limits of EBL

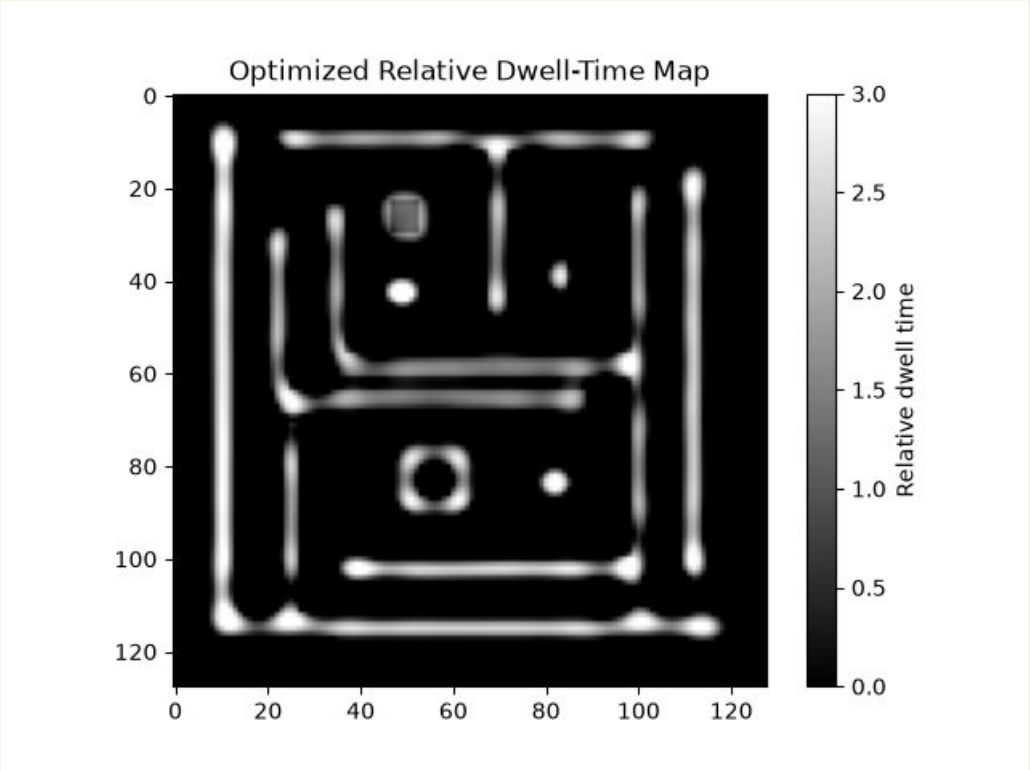
- Increasing beam energy(to 200 keV) narrows the Point Spread Function, this reduces scattering radius
- Ultra high voltages exponentially increase costs of equipment and operation
- High energy beams can physically damage sensitive resist



Our Algorithm

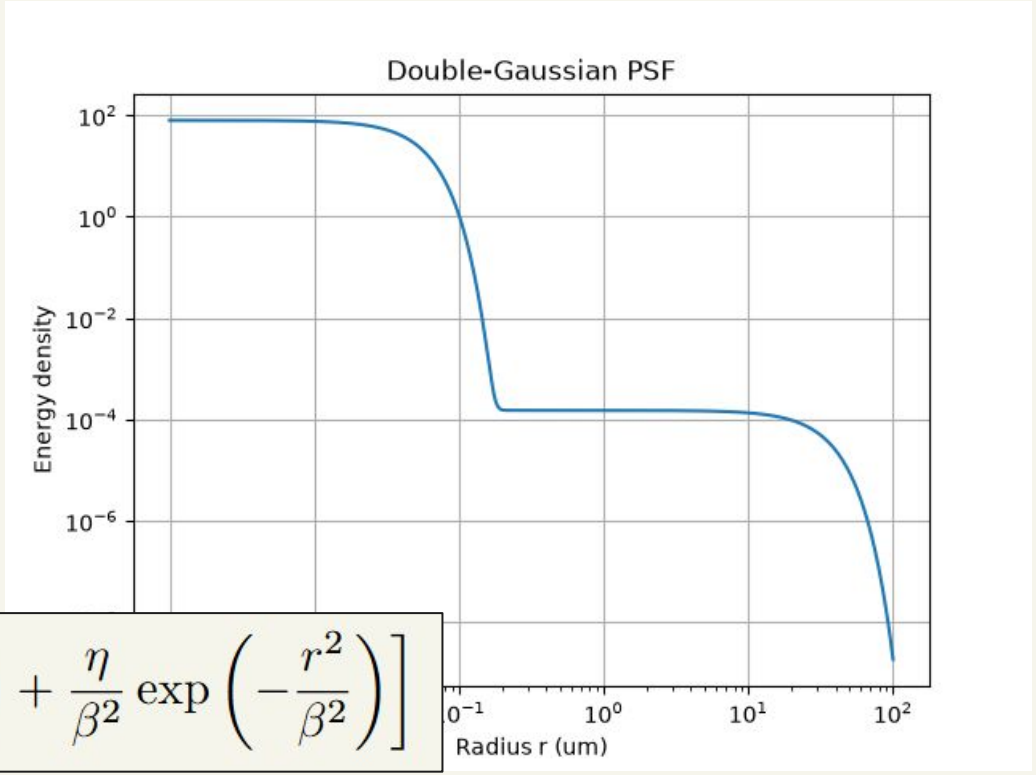
Iterative Exposure Pattern Adjustment

- Better exposure patterns:
  - Create better structures
  - Reduce the effects of the proximity effect



Proximity Effect Correlation in EBL

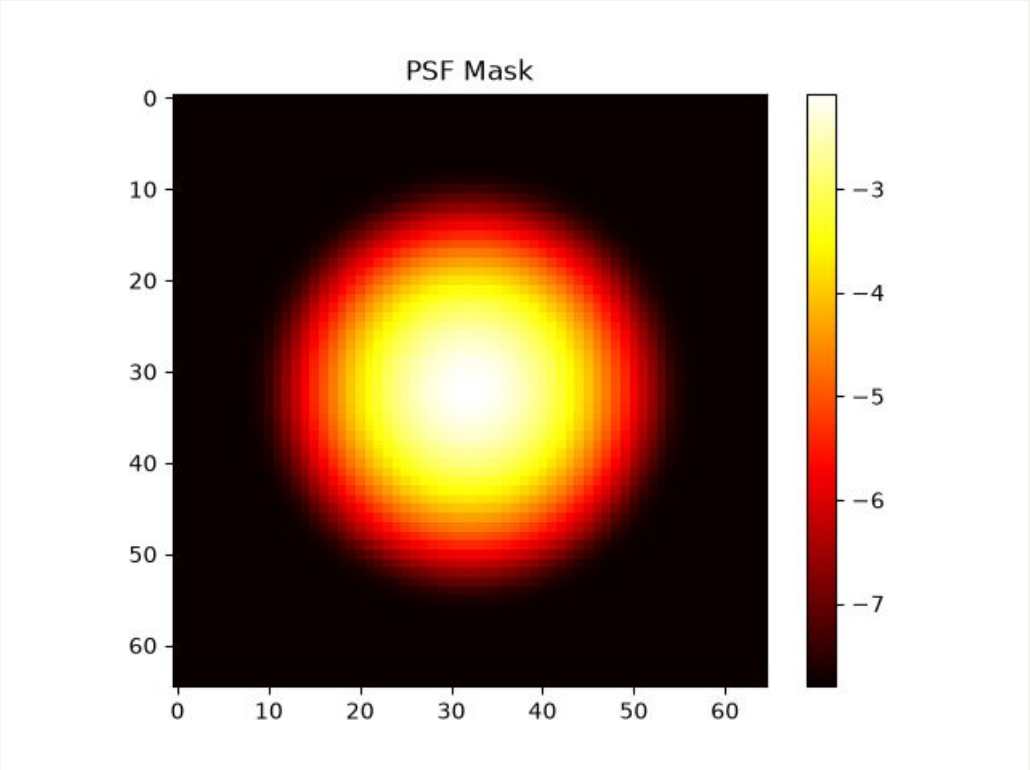
- The spread of electrons can be linearly modelled by the following equation
- Comprised of two gaussian functions to account for forward and back scattering



$$f(r) = \frac{1}{\pi(1+\eta)}\left[\frac{1}{\alpha^2}\exp\left(-\frac{r^2}{\alpha^2}\right) + \frac{\eta}{\beta^2}\exp\left(-\frac{r^2}{\beta^2}\right)\right]$$

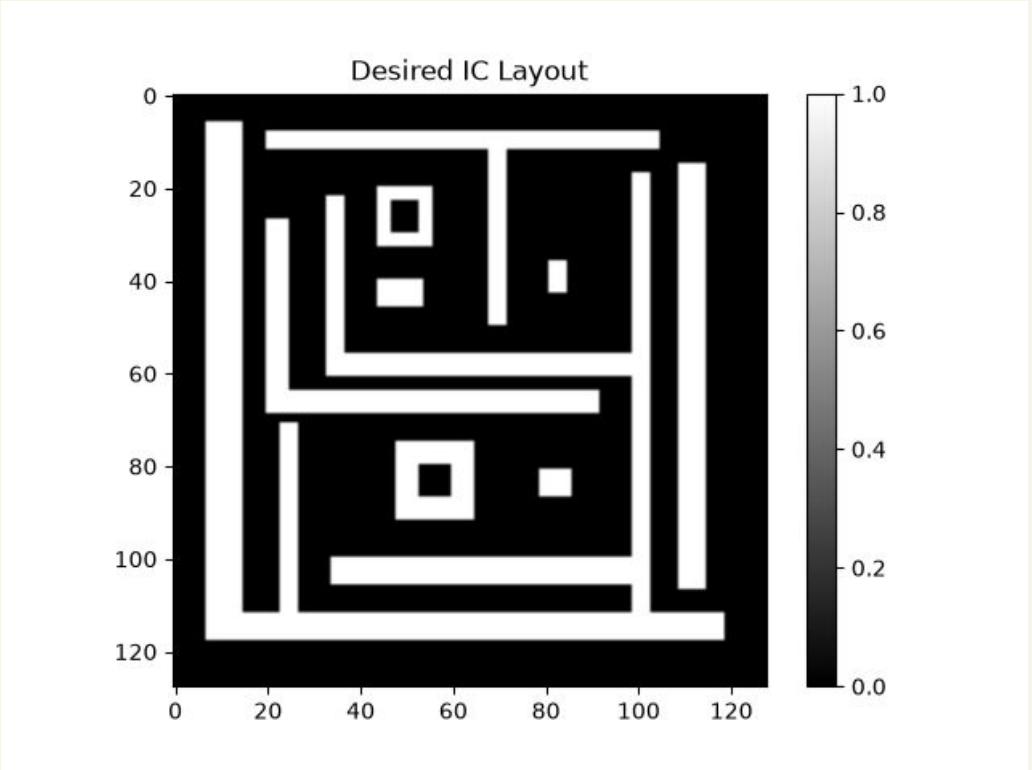
PSF Modeling

- That linear model is then extrapolated into a PSF
- Represents the spread of energy from the center point



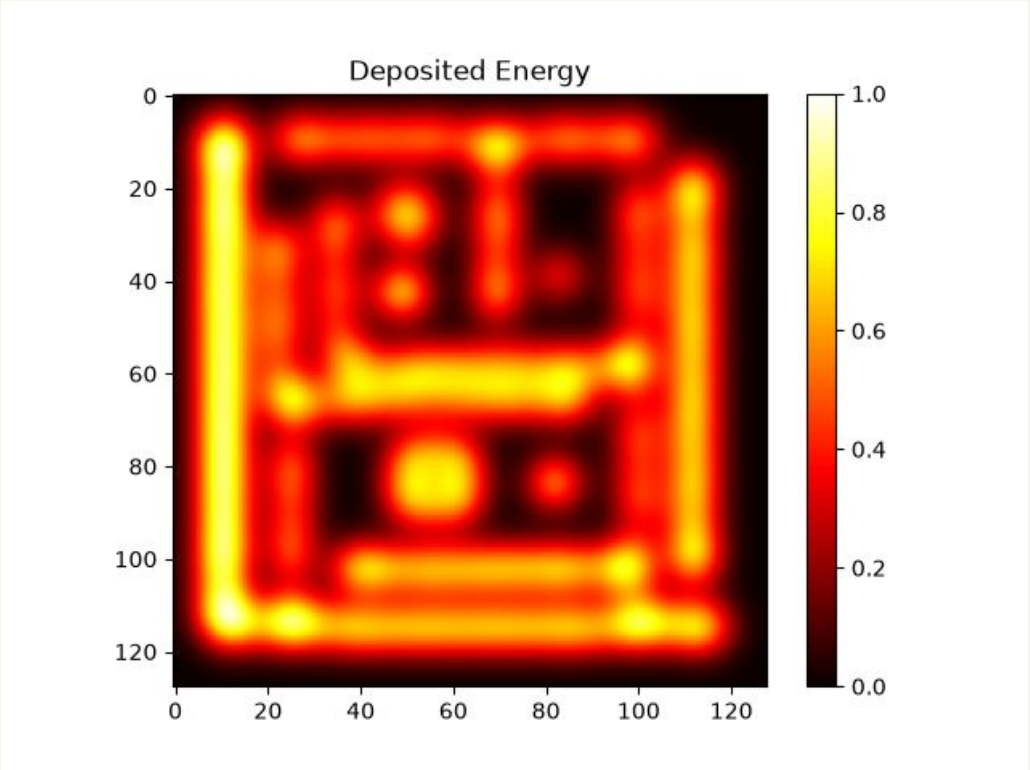
IC Layout

- Starting with a desired layout, the exposure pattern needs adjusting before fabricating structures



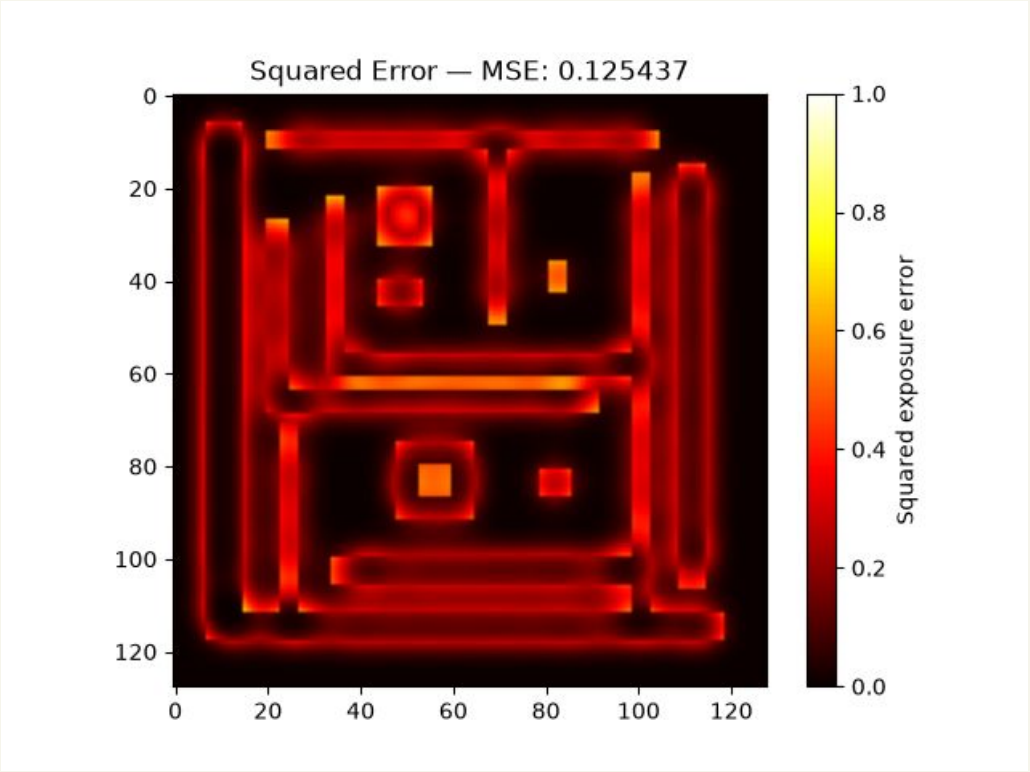
Convolution

- Simulate electron dispersion by convolving the PSF with the IC



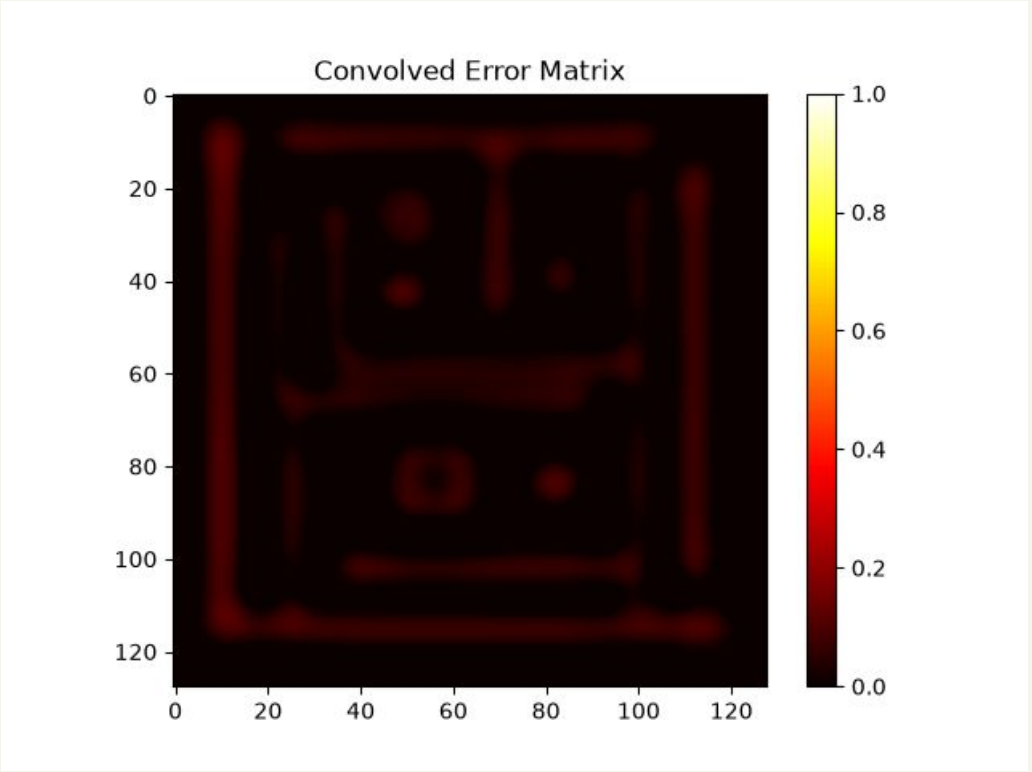
Mean Squared Error

- Subtract the convolved IC from the original
- The MSE is the average error across the entire image



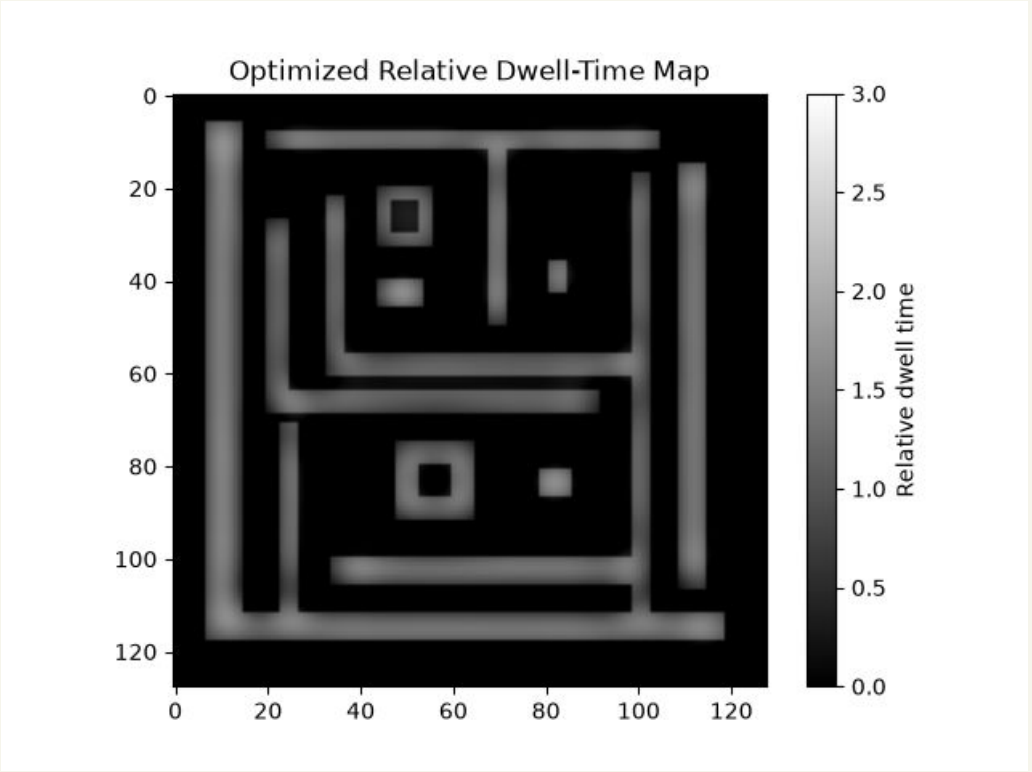
Convolved Error

- Convole the error with PSF
- 2nd convolution assigns blame for why each pixel has its error



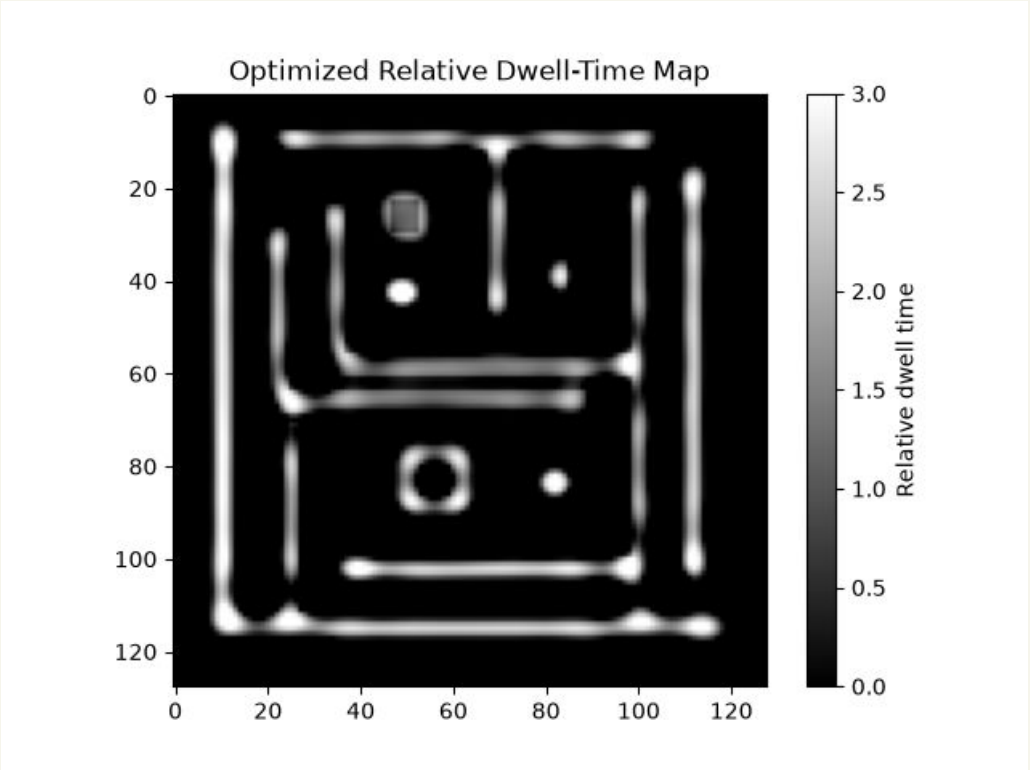
Update Layout

- Adjust layout based on the convolved error
- Negative convolved error = more exposure
- Positive convolved error = less exposure

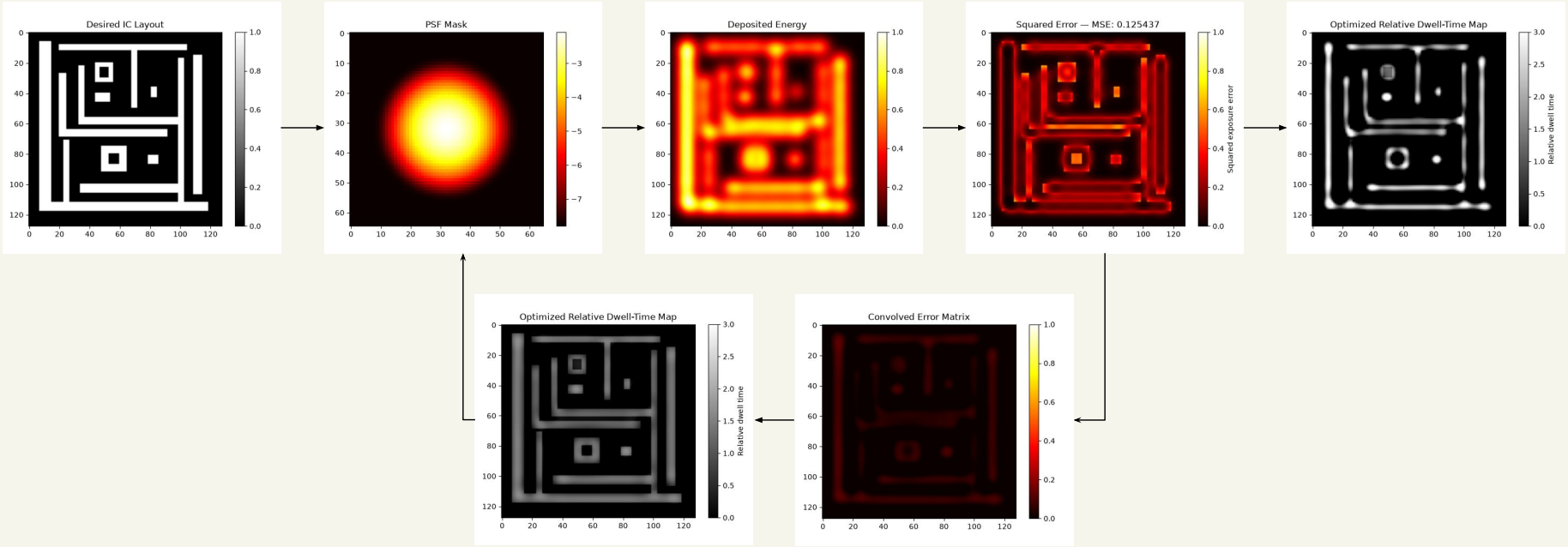


End Result

- After many iterations, the end result is an exposure pattern that minimizes error



Program Flow



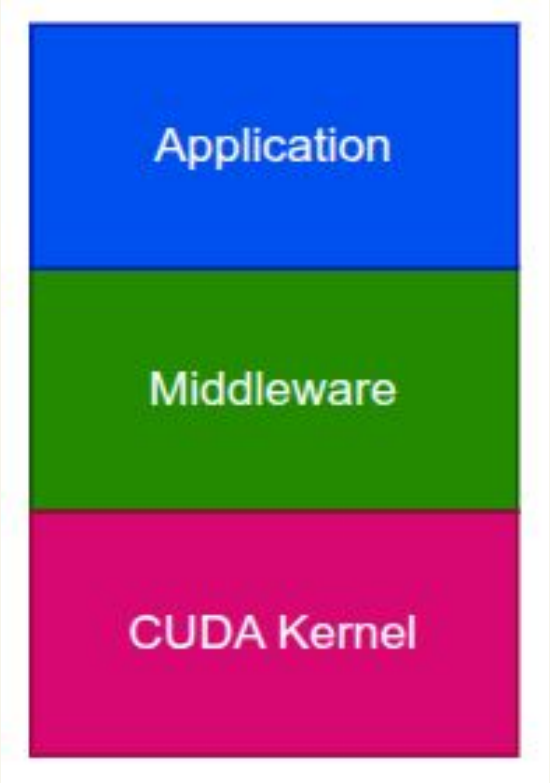
System Architecture and Design

System Design Decisions

- This system was designed with modularity in mind.
- Each layer represents a different responsibility in the system.

Layer Description:

- Application: Handles algorithm level logic
- Middleware: Abstraction layer for executing core logic
- CUDA Kernel: Logic being executed in parallel on GPU



System Architecture

Three components were developed in order to implement the Dwell Time EBL algorithm:

1. **Deposited Energy Calculations:** Calculates the deposited energy by convolving the current dwell time map with the PSF.

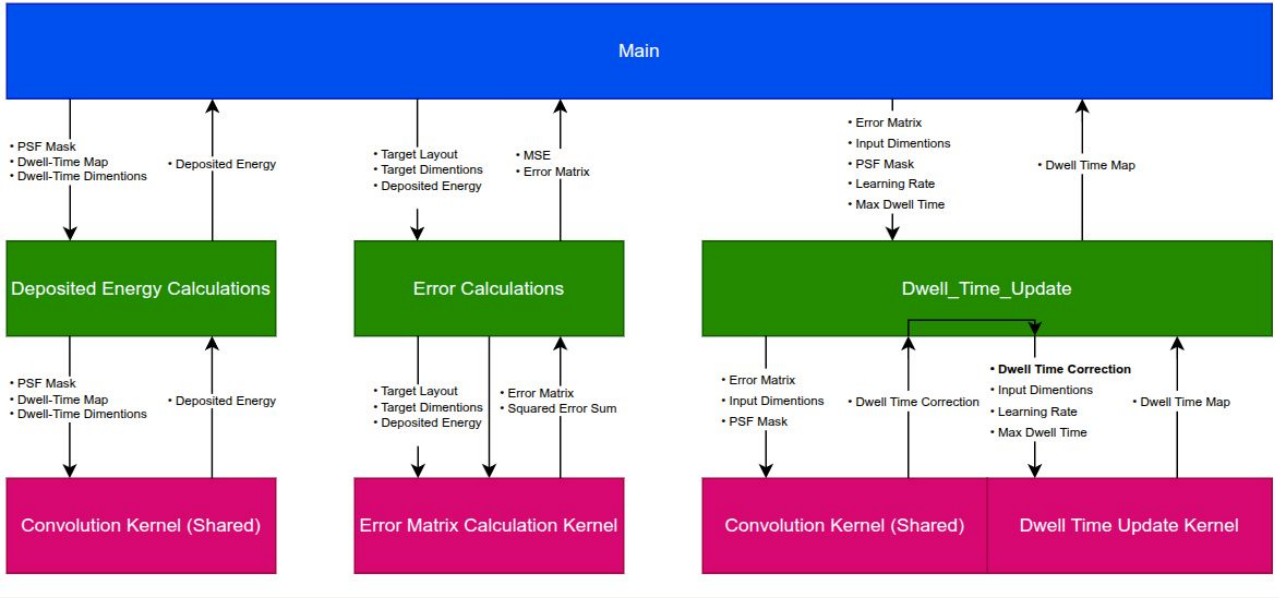
2. **Error Calculations:** Compares the deposited energy with the target layout to generate the error matrix and calculate MSE.

3. **Dwell Time Update:** Uses the error matrix to calculate a dwell time correction and update the dwell time map.

Note: Convolution kernel is generic since both deposited energy and dwell time calculations require it.

System Interfacing

- Being able to follow the data through the system is important.
- With the layered design in place, each component is shown with an input and output which shows how they will interface with each other.

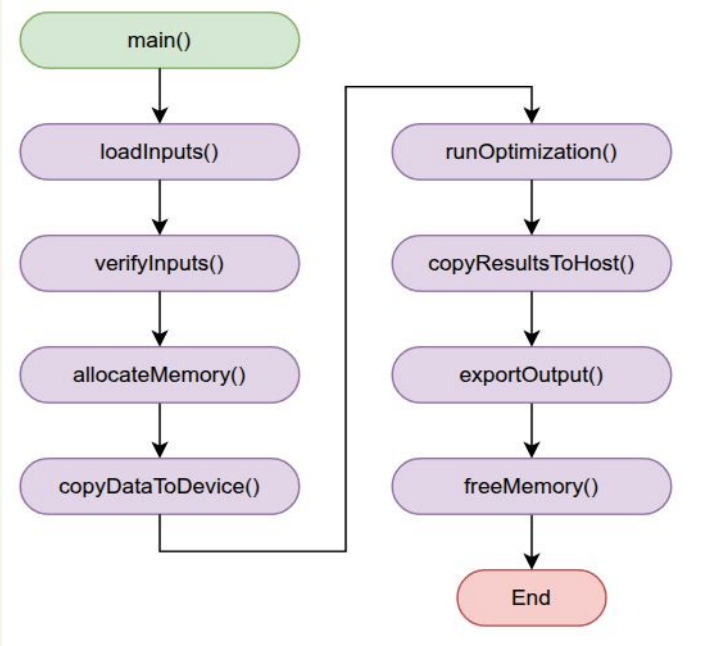


CUDA Implementation

Main Application Logic

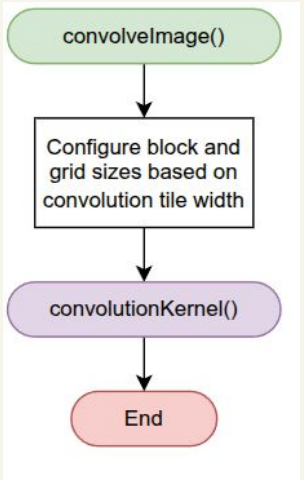
Main handles the overall application execution:

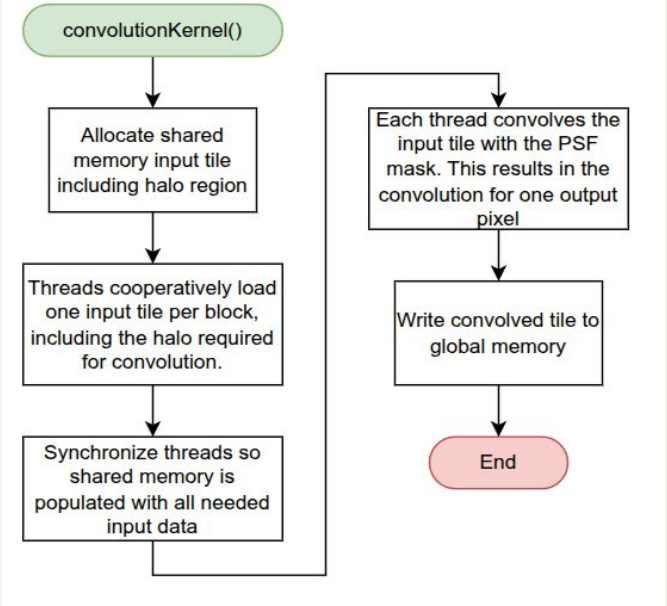
- Handling input data
- Allocating memory
- Transferring data from the host to the GPU
- **The algorithm itself**
  - Includes logging run time parameters
- Transferring data from the GPU back to the host
- Storing results on file system
- Deallocating memory



Deposited Energy

- In order to calculate the deposited energy, the current dwell time map needs to be convolved with the PSF mask.
- The tiling approach was used to allow each block to convolve a tile of the current dwell time map while still utilizing shared memory
- Cooperative loading was used to efficiently store the needed current dwell time map data into shared memory





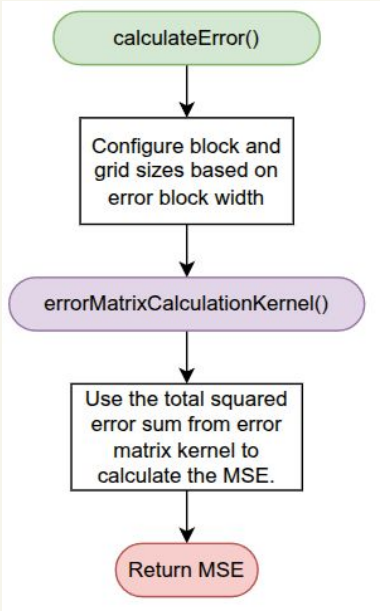
Error Matrix Calculation

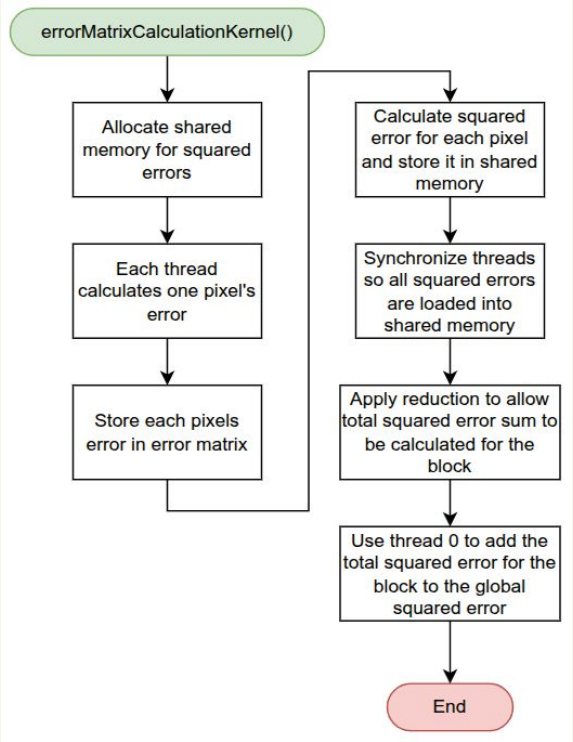
For calculating the error, two things need to be done:

1. Calculating the total squared error and error matrix between the target layout and the deposited energy

2. Using the total squared error to calculate the MSE

Reduction is used in the kernel to calculate the total squared error sum for each block.



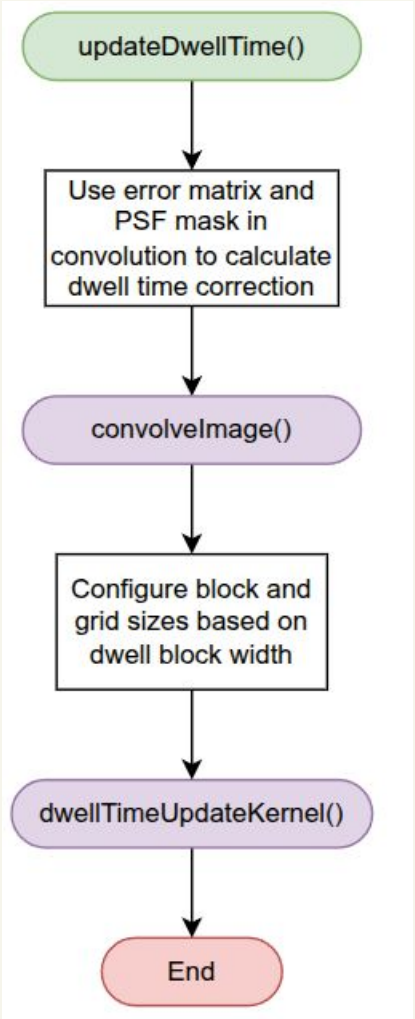


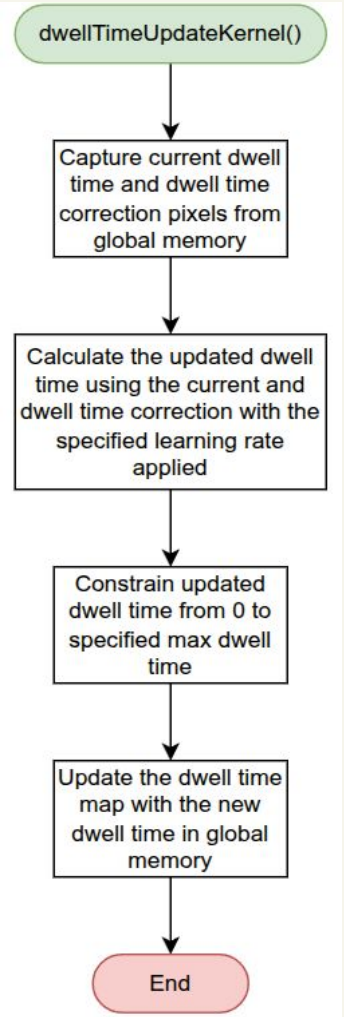
Dwell Time Update

Updating the dwell time is done in two steps:

1. Convolving the error matrix with the PSF mask. This calculates how much the current dwell time map needs to be corrected

2. Calculating and updating the current dwell time map using the dwell time map correction along with a specified learning rate. Conceptually it's:
    a. Current dwell time + (dwell time correction * learning rate)





Benchmarking

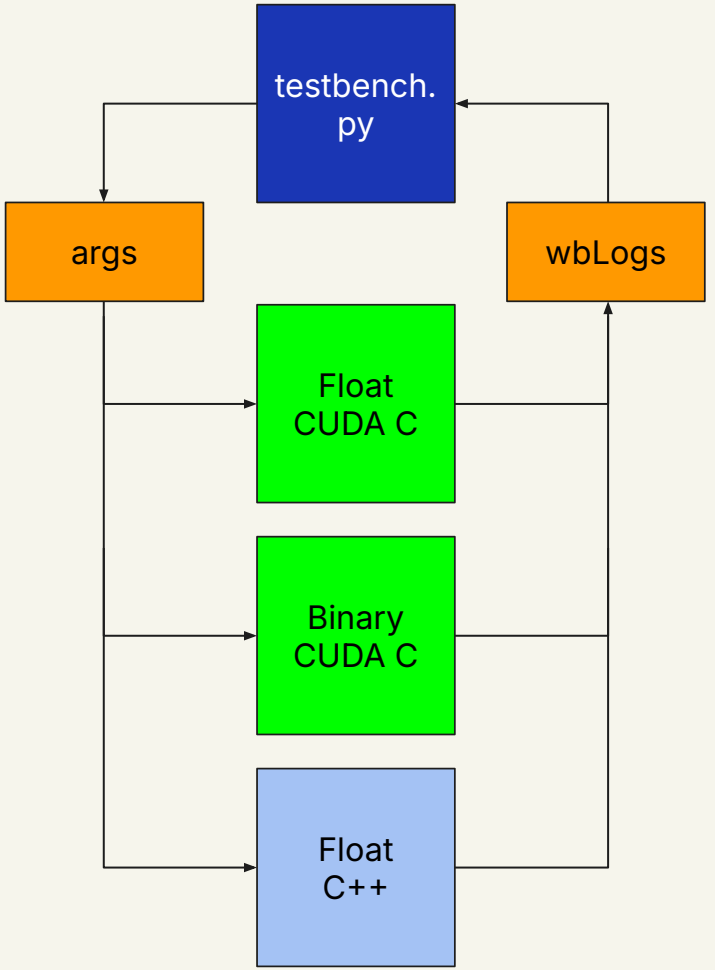
Methods

- All tests were performed on Windows 10 with
  - i7-7700 four core CPU
  - GTX 1660 GPU with 1,408 CUDA cores, 6 Gb of VRAM
  - 16Gb of RAM

- Three different approaches were tested
  - CUDA C accelerated floating point implementation
  - A Python implementation of the floating point algorithm that is logically and mathematically identical to the CUDA C version
  - CUDA C accelerated Binary implementation written by C. Vang, O. Yee, N. Kulkarni, and S. Alawneh
  - C++ sequential floating point implementation, logically and mathematically identical to the CUDA C version

Testbench

- First testbench runs all programs with the same arguments
- All programs are modified to receive the arguments
- All programs use libwb to save output
- Afterwards testbench synthesizes the results into .csv



Results

Execution Time

- Compared to a single core, 14-51 times faster
- Compared to the binary implementation roughly the same speed to two times slower scaling with input size

TABLE I
FLOATING POINT VS SEQUENTIAL EXECUTION TIME

Dataset	Resolution (pixels)	Execution Time (ms)				
		Floating Point	Binary	C++	Python	Float to Python
IC 0	128 × 128	267.8	288.8	11,576.1	3,830.8311	14.3×
IC 1	256 × 256	458.5	310.4	53,491.7	15,661.5643	34.2×
IC 2	512 × 512	1,153.8	590.1	221,460.9	59,160.3694	51.3×
IC 3	128 × 128	280.0	262.9	11,947.2	3,387.9547	12.1×
IC 4	256 × 256	464.3	300.2	50,863.2	14,537.1875	31.3×
IC 5	512 × 512	1,135.8	541.1	210,638.4	59,942.7259	52.8×
IC 6	128 × 128	265.3	248.4	10,846.0	3,818.4288	14.4×
IC 7	256 × 256	454.2	303.6	49,068.7	15,418.5721	33.9×
IC 8	512 × 512	1,125.6	556.5	207,308.7	58,682.5059	52.1×

MSE

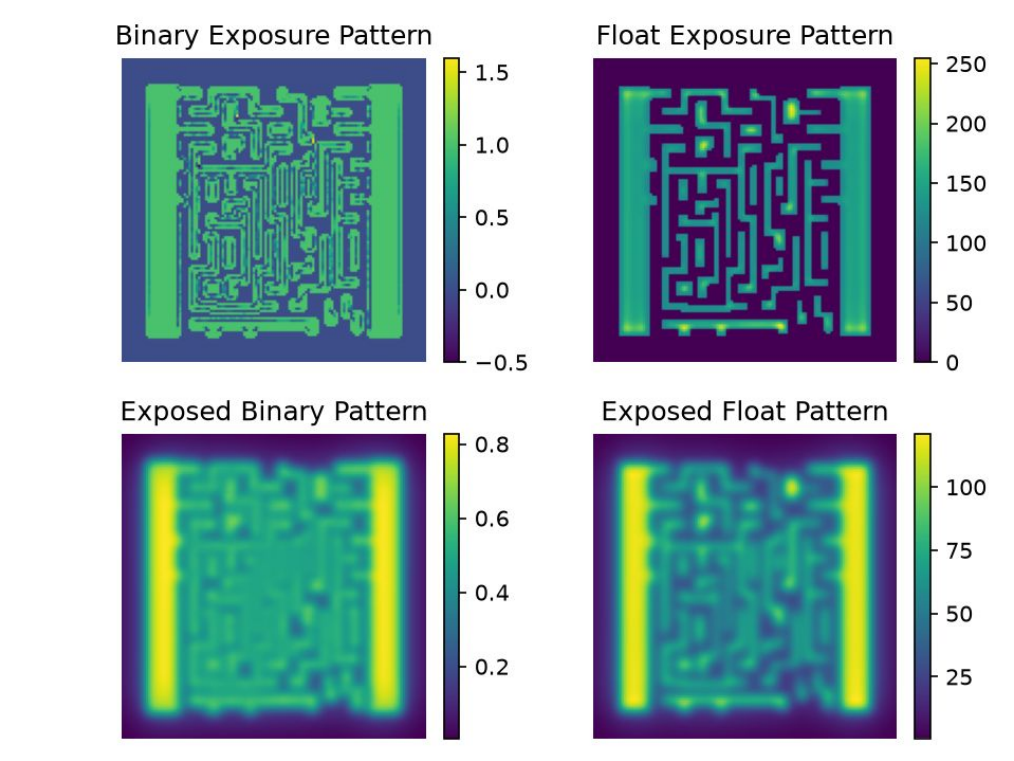
- Floating point implementation achieves 3-5 times better MSE

TABLE II
FLOATING POINT TO BINARY MSE WITH MULTIPLICATIVE INCREASE

Dataset	Resolution (pixels)	Final MSE		Float to Binary Increase
		Floating Point	Binary	
IC 0	128 × 128	0.063711	0.289094	4.54×
IC 1	256 × 256	0.054812	0.308273	5.62×
IC 2	512 × 512	0.051542	0.274517	5.33×
IC 3	128 × 128	0.076721	0.289954	3.78×
IC 4	256 × 256	0.072239	0.347875	4.82×
IC 5	512 × 512	0.061978	0.328938	5.31×
IC 6	128 × 128	0.108890	0.331472	3.04×
IC 7	256 × 256	0.109603	0.306051	2.79×
IC 8	512 × 512	0.087659	0.303314	3.46×

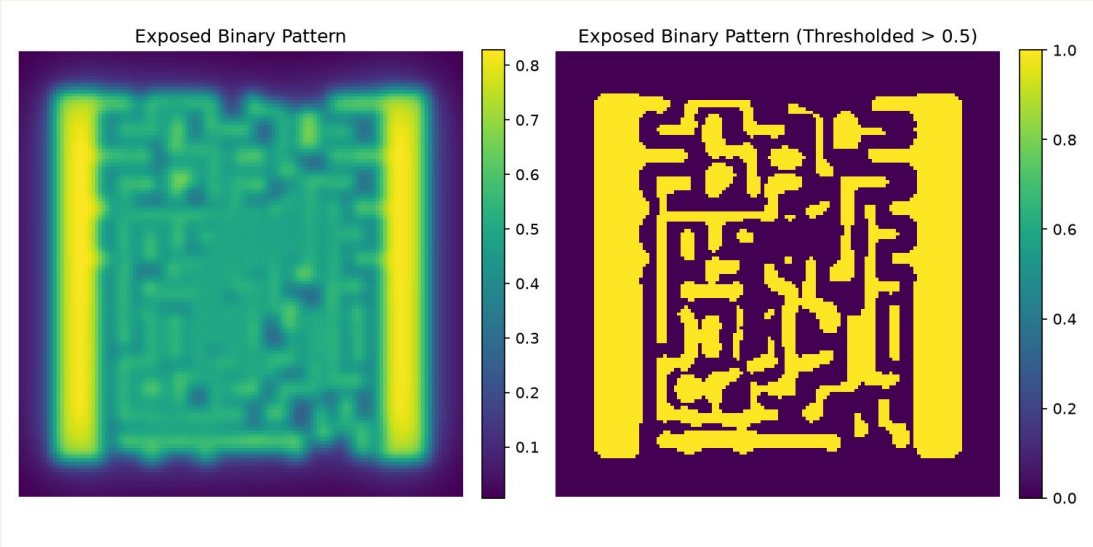
Structure Separation

- After exposure, structures in the floating point implementation are much more stable than binary



Structure Separation

- Threshold function ignores crucial detail in the exposed image
- Overexposed gaps have values between .4 and .5 and fail to appear after thresholding but still blur the final result



Conclusion

Conclusion

- Using CUDA C, our algorithm performed 14-51 times faster than an identical single core implementation
- Greatly increased structure definition without greatly increased execution time

References

1. V. R. Manfrinato, L. Zhang, D. Su, H. Duan, R. Hobbs, E. Stach, and K. K. Berggren, *Resolution limits of electron-beam lithography toward the atomic scale*
2. V. R. Manfrinato, J. Wen, L. Zhang, Y. Yang, R. G. Hobbs, B. Baker, D. Su, D. Zakharov, N. J. Zaluzec, D. J. Miller, E. A. Stach, and K. K. Berggren, *Determining the resolution limits of electron-beam lithography: direct measurement of the point-spread function*
3. M. A. Mohammad, T. Fito, J. Chen, S. Buswell, M. Aktary, S. K. Dew, and M. Stepanova, *The Interdependence of Exposure and Development Conditions when Optimizing Low-Energy EBL for Nano-Scale Resolution*
4. W. Yao, H. Xu, H. Zhao, M. Tao, and J. Liu, *Fast and accurate proximity effect correction algorithm based on pattern edge shape adjustment for electron beam lithography*
5. Q. Mao, J. Zhu, X. Cheng, and Z. Wang, *Proximity effect correction in electron beam lithography using a composite function model of electron scattering energy distribution*
6. C. Vang, O. Yee, N. Kulkarni, and S. Alawneh, *Proximity effect correction in electron beam lithography using CUDA to Increase Efficiency*